Практическая работа
Управление пакетами

Цели работы:

- изучение принципов управления пакетами в операционных системах семейства GNU/Linux
- освоение практических навыков работы с пакетными менеджерами APT и dpkg
- изучение различных типов пакетов (deb, метапакеты, snap, виртуальные).
- получение опыта создания собственного .deb-пакета, настройки локального репозитория и установки пакетов из него

Теоретические сведения

Система управления пакетами в операционных системах семейства GNU/Linux представляет собой комплекс программных средств, предназначенных для автоматизации процессов установки, обновления, конфигурации и удаления программного обеспечения. Данная система функционирует на основе концепции пакетов - стандартизированных архивных файлов, содержащих исполняемые коды, метаданные и скрипты управления. Важным механизмом, обеспечивающим целостность программной среды, является разрешение зависимостей - автоматическое определение и инсталляция вспомогательных библиотек и компонентов, необходимых для функционирования целевого приложения.

Типы пакетов в современных Linux-дистрибутивах

Обычные .deb-пакеты представляют собой классический формат распространения программ. Это бинарные пакеты, содержащие скомпилированные программы, библиотеки, конфигурационные файлы и метаданные. Они распространяются через официальные репозитории, управляются системой APT/dpkg и тесно интегрированы в общую систему зависимостей. При установке такие пакеты размещают свои файлы в стандартных каталогах файловой системы, что обеспечивает единообразие и предсказуемость среды выполнения.

Метапакеты выполняют особую организационную роль в пакетной системе. В отличие от обычных пакетов, они не содержат собственных файлов программ, а включают только зависимости других пакетов. Основное назначение метапакетов - группировка логически связанного программного обеспечения. Например, пакет ubuntu-desktop объединяет все компоненты, необходимые для рабочей среды Ubuntu. Это значительно упрощает установку комплексного ПО и обеспечивает согласованность рабочих сред, позволяя пользователям устанавливать целые программные комплексы одной командой.

Snap пакеты представляют собой современный формат распространения приложений, основанный на технологии контейнеризации. Их архитектура строится на двух фундаментальных принципах. Первый - это строгая изоляция, достигаемая за счёт запуска приложения в защищённой среде с ограниченным доступом к системным ресурсам. Второй ключевой принцип - кросс-дистрибутивность. Благодаря тому, что snap-пакет включает в себя все необходимые для работы зависимости, один и тот же собранный пакет может быть установлен и будет корректно работать на различных дистрибутивах Linux. Это решает проблему фрагментации экосистемы Linux, избавляя разработчиков от необходимости создавать и поддерживать отдельные сборки для каждого дистрибутива (.deb, .rpm и др.), а пользователям гарантирует доступ к единой версии приложения независимо от выбора дистрибутива.

Flatpak пакеты - еще одна современная технология контейнеризации приложений, которая обеспечивает изоляцию и кроссплатформенность. В отличие от Snap, где каждый пакет содержит полный набор библиотек. Flatpak использует общие рантаймы - базовые наборы библиотек, которые могут использоваться сразу несколькими приложениями. Безопасность обеспечивается с помощью порталов доступа - специальных интерфейсов,

предоставляющих изолированному приложению контролируемый доступ к системным ресурсам.

Таблица 1. Сравнение типов пакетов

Параметр	Обычный .deb	Метапакет	Snap	Flatpak
содержимое	файлы программ + зависимости	только зависимости	программа + все зависимости в контейнере	приложение + общие рантаймы
изоляция	нет	нет	полная	полная (с порталами доступа)
размер	зависит от программы	минимальный	большой	средний (за счет общих рантаймов)
зависимости	разрешаются системой	определяют группу пакетов	автономные	использует общие рантаймы
путь установки	/usr, /etc	-	/snap/	/var/lib/flatpak/, ~/.local/share/flatpak/
управление	APT/dpkg	APT/dpkg	snapd	flatpak

Структура .deb пакетов

Формат .deb представляет собой стандартизированный архивный формат пакетов в дистрибутивах Debian и Ubuntu. Данный формат использует структуру архива ar для объединения нескольких компонентов в единый файл. Основу пакета составляют три элемента:

- файл debian-binary, содержащий версию формата пакета
- архив control.tar.gz с мета-информацией и скриптами управления
- архив data.tar.gz непосредственно с файлами программного обеспечения.

В составе control.tar.gz располагаются критически важные элементы управления пакетом:

- файл control содержит основную мета-информацию, такую как имя пакета, версия, архитектура, зависимости и описание
- скрипты preinst и postinst выполняются соответственно до и после установки пакета, обеспечивая правильную интеграцию с системой
- скрипты prerm и postrm управляют процессом удаления пакета, выполняя необходимые очистки и изменения конфигурации
- файл conffiles идентифицирует конфигурационные файлы, которые должны сохраняться при обновлении пакета.

Архив data.tar.gz содержит иерархическую структуру файлов, которая точно соответствует их целевому расположению в файловой системе. При установке пакета эти файлы распаковываются в соответствующие каталоги, такие как /usr/bin для исполняемых файлов, /usr/lib для библиотек и /etc для конфигурационных файлов.

Для анализа содержимого ***.deb*** пакета без его установки применяются команды ***dpkg*** и ***ar.*** Команды ***dpkg -c*** и ***dpkg -I*** позволяют просмотреть файлы данных и мета-информацию соответственно. Для работы с внутренней структурой архива используются ***ar -t*** (просмотр состава) и ***ar -x*** (извлечение компонентов), что обеспечивает полный контроль над содержимым пакета перед установкой.

Двухуровневая система управления пакетами

В основе управления программным обеспечением в **deb**-дистрибутивах Linux лежит двухуровневая архитектура, состоящая из низкоуровневого менеджера **dpkg** и высокоуровневого менеджера **APT** (Advanced Package Tool). Такой подход обеспечивает

баланс между контролем над отдельными пакетами и автоматизированным управлением программными комплексами.

1. Низкоуровневый менеджер **dpkg**

представляет собой базовый пакетный менеджер, функционирующий на уровне отдельных файлов пакетов формата .deb.

- **dpkg** работает исключительно с локальными файлами пакетов, не взаимодействуя с сетевыми репозиториями
- при выполнении команды установки пакета
  dpkg -i package.deb **(**или ***dpkg –install package.deb*****)**
  dpkg последовательно выполняет:
  - верификацию целостности пакета (проверка целостности архива .deb, валидация цифровой подписи пакета (при наличии), контроль корректности внутренней структуры пакета);
  - анализ метаданных (извлечение информации из файла control, парсинг декларированных зависимостей (поля Depends, Pre-Depends), проверку конфликтов и замен (поля Conflicts, Replaces));
  - проверку наличия в системе требуемых зависимостей, при обнаружении неудовлетворенных зависимостей операция приостанавливается, пакет помечается как "установленный с ошибками";
  - распаковку содержимого пакета (распаковка файлов данных в целевую файловую систему с сохранением прав доступа, размещение исполняемых файлов, библиотек, документации согласно иерархии FHS);
  - выполнение скриптов управления (исполнение pre-installation скрипта (preinst) перед распаковкой, запуск post-installation скрипта (postinst) после успешной экстракции);
  - обновление базы данных пакетов в /var/lib/dpkg/status.

Основные команды **dpkg:**

- ***dpkg -i package.deb*** # установка пакета
- ***dpkg -r package_name*** # удаление пакета (конфигурационные файлы сохраняются)
- ***dpkg -P package_name*** # полное удаление с конфигами
- ***dpkg -l*** # список всех установленных пакетов
- ***dpkg -s package_name*** # информация о конкретном пакете
- ***dpkg -L package_name*** # список файлов пакета
- ***dpkg -S /path/to/file*** # какой пакет владеет файлом
- ***dpkg --configure -a*** # конфигурировать все незавершенные пакеты
- ***dpkg --audit*** # проверить целостность базы данных

2. Высокоуровневый менеджер **APT**

представляет собой надстройку над **dpkg**, обеспечивающую:

- работу с репозиториями, структура которых описана в конфигурационных файлах /etc/apt/sources.list, /etc/apt/sources.list.d/;
- интеллектуальное разрешение зависимостей (автоматический анализ, загрузка и установка всех необходимых зависимостей с помощью построения графа зависимостей);
- кеширование метаданных: команда ***apt update*** синхронизирует локальный кеш информации о доступных пакетах с данными репозиториев.

APT обеспечивает различные сценарии управления пакетами:

- ***apt install*** - установка с автоматическим разрешением зависимостей
- ***apt remove*** - деинсталляция пакета с сохранением конфигурационных файлов
- ***apt purge*** - полная элиминация пакета вместе с конфигурацией
- ***apt --fix-broken install*** - автоматическое восстановление целостности пакетной системы.

3. Взаимодействие **APT** и **dpkg**

APT и **dpkg** работают в тесном симбиозе, формируя единую систему управления пакетами в deb-дистрибутивах. **APT** функционирует на высоком уровне абстракции, осуществляя взаимодействие с сетевыми репозиториями и управляя деревьями зависимостей, **dpkg** оперирует на низком уровне, выполняя непосредственные операции с локальными файлами пакетов и осуществляя физическую установку программных компонентов в файловую систему.

Таблица 2. Взаимодействие APT и dpkg

Операция	**APT**	**dpkg**
установка пакета	загружает пакет и все зависимости из репозиториев, анализирует дерево зависимостей	физически устанавливает каждый .deb файл в систему, распаковывает архивы
обновление системы	определяет какие пакеты требуют обновления, скачивает новые версии из репозиториев	последовательно применяет обновления к каждому пакету, выполняет скрипты обновления
удаление пакета	определяет какие файлы и конфигурации относятся к пакету, проверяет зависимости	выполняет скрипты удаления и физически удаляет файлы из файловой системы
работа с базами данных	использует кеш информации о доступных пакетах из репозиториев	поддерживает актуальную базу установленных пакетов (/var/lib/dpkg/status)
разрешение зависимостей	автоматически находит и разрешает все транзитивные зависимости	сообщает о нарушениях зависимостей при прямой установке .deb файлов

Типы зависимостей в .deb пакетах

Система управления пакетами в deb-системах использует различные типы зависимостей, которые определяют степень важности связи между пакетами:

- depends - обязательные зависимости, пакеты, без которых установленный пакет не сможет работать
- recommends - рекомендуемые пакеты, расширяющие функциональность основного пакета
- suggests - опциональные зависимости, пакеты, предоставляющие дополнительные возможности, но не обязательные для базовой работы
- enhances - пакеты, которые улучшают функциональность других пакетов, но не являются их прямыми зависимостями
- conflicts - конфликтующие пакеты, не могут быть установлены одновременно с данным пакетом
- breaks - пакеты, которые нарушают работу данного пакета при установке.

Практические задания

Задание 1. Идентификация типов пакетов

Исследуйте пакеты nano, ubuntu-minimal, firefox, core22, chromium и определите их тип.

Для каждого пакета выполните:

apt show <имя_пакета>
dpkg -L <имя_пакета> 2>/dev/null | head -10
snap list <имя_пакета> 2>/dev/null

На основании полученных данных заполните таблицу:

Пакет	Тип	Размер	Зависимости	Содержит файлы
nano				
ubuntu-minimal				
firefox				
core22				
chromium				

Подсказки для определения типа пакета:

Тип пакета	Признаки
Обычный .deb	• apt show возвращает корректные значения размера и зависимостей • dpkg -L выводит значительное количество файлов, расположенных в стандартных каталогах (/usr/bin, /etc, /usr/lib и др.) • snap list не содержит информации о пакете
Метапакет	• apt show отображает незначительный размер и развёрнутый список обязательных зависимостей • dpkg -L возвращает ограниченный набор файлов (как правило, только документацию в /usr/share/doc) • snap list не содержит информации о пакете
Переходный (на snap)	• apt show указывает версию, содержащую идентификатор snap (например, 1:1snap*), и малый размер • dpkg -L выводит лишь служебные файлы документации, исполняемые файлы отсутствуют • после установки такого пакета snap list может отображать соответствующий snap-пакет (если он был установлен автоматически)
Snap-пакет	• apt show завершается ошибкой или сообщением об отсутствии пакета в репозиториях APT • dpkg -L не возвращает данных (или выдаёт ошибку) • snap list содержит запись о пакете с указанием версии, ревизии и канала
Виртуальный	• apt show информирует о виртуальной природе пакета и отсутствии кандидата для установки • dpkg -L не возвращает данных • snap list может содержать или не содержать запись о snap-пакете, предоставляющем данную функциональность

Вопросы для отчёта по заданию:

1. Какие типы пакетов вы обнаружили? Перечислите их.
2. Чем отличается вывод ***apt show*** для обычного пакета и метапакета?
3. Почему для firefox команда dpkg -L показывает мало файлов, но браузер работает?
4. Что такое виртуальный пакет? Приведите пример из задания.
5. Как определить, что пакет является snap-пакетом, используя данные команды?

Задание 2. Анализ метапакетов в системе

2.1 Найдите все метапакеты в вашей системе:

apt-cache show $(dpkg-query -Wf '${Package}\n') | awk '$1 == "Package:" { pkg = $2 }; $1 == "Section:" && $2 ~ /metapackage/ { print pkg }'

Эта команда анализирует все пакеты в системе и выводит те, у которых в поле "Section" указано "metapackage".

Альтернативная команда для поиска метапакетов:

apt-cache search --names-only ".*" | grep -i "meta"

Убедитесь, что в списке есть ubuntu-standard. Если его нет, выберите для исследования другой метапакет из полученного списка.

2.2 Анализ метапакета ubuntu-standard

Получите общую информацию о метапакете:

apt show ubuntu-standard

Проанализируйте зависимости метапакета:

apt-cache depends ubuntu-standard

Определите типы зависимостей (используйте LC_ALL=C перед командой для гарантии работы независимо от локализации системы):

apt-cache show ubuntu-standard | grep Depends
apt-cache show ubuntu-standard | grep Recommends
apt-cache show ubuntu-standard | grep Suggests

Сколько зависимостей типа Depends имеет метапакет? Есть ли у него зависимости типов Recommends и Suggests?

2.3 Проверка состояния установки:

dpkg -l ubuntu-standard

Если метапакет установлен, определите, какие файлы он содержит:

dpkg -L ubuntu-standard

Если метапакет не установлен, установите его:

sudo apt install ubuntu-standard

и повторите предыдущий пункт.

2.4 Исследование взаимосвязей

Узнайте, какие пакеты зависят от ***ubuntu-standard:***

apt-cache rdepends ubuntu-standard

Выберите один из пакетов, который ***рекомендует ubuntu-standard*** (из вывода пункта 2.2). Если таких нет, выберите пакет ***sl.*** Установите его:

sudo apt install <имя_рекомендованного_пакета>
или
sudo apt install sl

Проанализируйте зависимости установленного пакета:

apt-cache depends <имя_установленного_пакета>

2.5 Управление зависимостями

Удалите установленный в пункте 2.4 пакет:

sudo apt remove <имя_установленного_пакета>

Проверьте, остались ли неиспользуемые зависимости:

sudo apt autoremove --dry-run

Если они есть, удалите их:

sudo apt autoremove

Вопросы для отчета по заданию:

1. Какие метапакеты вы обнаружили в системе?
2. Какие три типа зависимостей (Depends/Recommends/Suggests) присутствуют у ubuntu-standard? Приведите по одному примеру каждого типа.
3. Сколько файлов содержит метапакет ubuntu-standard? Почему их так мало?
4. Был ли `ubuntu-standard` уже установлен в системе? Если да, что это говорит о вашей системе?
5. Какой пакет вы установили дополнительно? Какие у него были зависимости?
6. Что произошло при удалении дополнительного пакета? Удалились ли его зависимости?
7. Какие практические преимущества дает использование метапакетов при развертывании одинаковых систем на нескольких компьютерах?

Задание 3. Создание простейшего .deb пакета

Перед выполнением задания выберите имя для своего пакета. Оно должно быть написано латиницей, без пробелов. Используйте вашу фамилию (транслитом) и слово ***firstpackage***, например: ***ivanovfirstpackage***. Везде в командах вместо ***myfirstpackage*** подставляйте выбранное вами имя.

3.1 Создайте рабочую директорию с правильной структурой:

```
mkdir -p ~/myfirstpackage/DEBIAN
mkdir -p ~/myfirstpackage/usr/local/bin
mkdir -p ~/myfirstpackage/etc/mypackage
```

3.2 Создайте скрипт

```
cat > ~/myfirstpackage/usr/local/bin/myscript << 'EOF'
#!/bin/bash
echo "Мой первый пакет работает успешно!"
echo "Текущая дата: $(date)"
echo "Версия пакета: 1.0"
EOF
chmod +x ~/myfirstpackage/usr/local/bin/myscript
```

3.3 Создайте файл control в каталоге DEBIAN

В этом файле указывается метаинформация о пакете. В поле ***Maintainer*** укажите свои фамилию и имя латиницей (например, ***Ivan Petrov***). При желании можно добавить email в угловых скобках: ***Ivan Petrov <ivan@example.com>.*** Остальные поля оставьте как в примере, но не забудьте заменить имя пакета в строке ***Package:***

```
cat > ~/myfirstpackage/DEBIAN/control << EOF
Package: myfirstpackage
Version: 1.0
Section: utils
Priority: optional
Architecture: all
Maintainer: Student <student@example.com>
Description: Мой первый тестовый пакет
 Это учебный пакет, созданный для освоения системы управления пакетами.
EOF
```

3.4 Соберите пакет:

```
dpkg-deb --build ~/myfirstpackage
```

3.5 Установите и протестируйте пакет:

установка:

```
sudo dpkg -i myfirstpackage.deb
```

проверка установки:

```
dpkg -l | grep myfirstpackage
```

проверка файлов пакета:
dpkg -L myfirstpackage
запуск скрипта:
myscript

Вопросы для отчета по заданию:

1. Какие файлы обязательно должны быть в .deb пакете?
2. Почему важно указывать архитектуру (Architecture) в control-файле?
3. Как удалить созданный пакет из системы?

Задание 4. Настройка локального репозитория

4.1 Установите инструменты для создания репозитория:
sudo apt install dpkg-dev
Пакет ***dpkg-dev*** содержит утилиты для работы с исходными пакетами и создания индексных файлов репозитория.
4.2 Создайте директорию для локального репозитория:
mkdir -p ~/local-repo
4.3 Поместите созданный вами пакет в репозиторий:
cp myfirstpackage.deb ~/local-repo/
4.4 Создайте индекс репозитория:
cd ~/local-repo
dpkg-scanpackages . /dev/null > Packages
Примечание: при желании можно сжать индекс командой gzip Packages, но в учебных целях рекомендуется оставить несжатый файл для упрощения отладки.
dpkg-scanpackages . /dev/null | gzip > Packages.gz

- ***dpkg-scanpackage***s сканирует текущий каталог (.) на наличие .deb файлов и создаёт описание пакетов.
- ***/dev/null*** используется вместо файла переопределений (override file), так как он нам не нужен.
- результат сжимается в ***Packages.gz,*** именно этот файл APT будет читать при обновлении.

4.5 Добавьте локальный репозиторий в систему ***:***
echo "deb [trusted=yes] file:$(pwd) ./" | sudo tee /etc/apt/sources.list.d/local-repo.list

- ***[trusted=yes]*** указывает APT доверять неподписанному локальному репозиторию
- ***$(pwd)*** получает текущий путь (полный путь к ***~/local-repo).***
- файл ***local-repo.list*** создаётся в каталоге ***sources.list.d.***

4.6 Обновите список пакетов и проверьте доступность пакета:
sudo apt update
apt policy myfirstpackage

Команда ***apt policy*** покажет, из какого репозитория доступен пакет и его версию. Убедитесь, что пакет виден.
4.7 Установите пакет из локального репозитория:
sudo apt install myfirstpackage

Если пакет уже был установлен ранее, появится сообщение, что он уже установлен. В этом случае можно сначала удалить его ***(sudo apt remove myfirstpackage***) и затем установить заново, чтобы убедиться, что установка происходит из локального репозитория.
4.8 Удалите локальный репозиторий после того, как работа будет зачтена преподавателем:
sudo rm /etc/apt/sources.list.d/local-repo.list
sudo apt update

Это очистит список источников и предотвратит случайное использование локального репозитория в будущем.

Вопросы для отчета по заданию:

1. Для чего нужна утилита **dpkg-scanpackages** и что она создаёт?

2. Что представляет собой файл Packages.gz и какую роль он играет в работе APT?
3. Почему в строке репозитория необходимо указывать параметр [trusted=yes]? Что произойдёт без него?
4. Как проверить, что локальный репозиторий успешно добавлен и пакет в нём доступен для установки?
5. Какие преимущества даёт использование локального репозитория по сравнению с установкой пакета напрямую через dpkg -i?
6. В каких ситуациях системному администратору может потребоваться создавать локальный репозиторий?
7. Как удалить локальный репозиторий из системы, чтобы он больше не использовался?

Дополнительное задание

- Добавьте в локальный репозиторий ещё один созданный вами пакет (например, второй версии). Обновите индекс (dpkg-scanpackages) и убедитесь, что APT видит оба пакета.
- Попробуйте подписать локальный репозиторий GPG-ключом и настроить его как trusted без параметра [trusted=yes].